

Problem Set 1

Applied Stats II

Due: February 11, 2026

Instructions

- Please show your work! You may lose points by simply writing in the answer. If the problem requires you to execute commands in `R`, please include the code you used to get your answers. Please also include the `.R` file that contains your code. If you are not sure if work needs to be shown for a particular problem, please ask.
- Your homework should be submitted electronically on GitHub in `.pdf` form.
- This problem set is due before 23:59 on Wednesday February 11, 2026. No late assignments will be accepted.

Question 1

The Kolmogorov-Smirnov test uses cumulative distribution to statistics test the similarity of the empirical distribution of some observed data and a specified PDF, and serves as a goodness of fit test. The test statistic is created by:

$$D = \max_{i=1:n} \left\{ \frac{i}{n} - F_{(i)}, F_{(i)} - \frac{i-1}{n} \right\}$$

where F is the theoretical cumulative distribution of the distribution being tested and $F_{(i)}$ is the i th ordered value. Intuitively, the statistic takes the largest absolute difference between the two distribution functions across all x values. Large values indicate dissimilarity and the rejection of the hypothesis that the empirical distribution matches the queried theoretical distribution. The p-value is calculated from the Kolmogorov- Smirnov CDF:

$$p(D \leq d) = \frac{\sqrt{2\pi}}{d} \sum_{k=1}^{\infty} e^{-(2k-1)^2\pi^2/(8d^2)}$$

which generally requires approximation methods (see Marsaglia, Tsang, and Wang 2003). This so-called non-parametric test (this label comes from the fact that the distribution of the test statistic does not depend on the distribution of the data being tested) performs

poorly in small samples, but works well in a simulation environment. Write an R function that implements this test where the reference distribution is normal. Using R generate 1,000 Cauchy random variables (`rcauchy(1000, location = 0, scale = 1)`) and perform the test (remember, use the same seed, something like `set.seed(123)`, whenever you're generating your own data).

As a hint, you can create the empirical distribution and theoretical CDF using this code:

```
1 # create empirical distribution of observed data
2 ECDF <- ecdf(data)
3 empiricalCDF <- ECDF(data)
4 # generate test statistic
5 D <- max(abs(empiricalCDF - pnorm(data)))
```

Response

The goal of this exercise is to implement the Kolmogorov–Smirnov test manually in R, assuming a normal reference distribution, and to apply it to 1,000 Cauchy random variables.

The first step consists of simulating the data in R according to the instructions. The function `set.seed()` ensures that the same pseudo-random numbers are generated each time the script is executed, guaranteeing the replicability of the results. We then generate 1,000 observations—assigned to `x`—from a Cauchy distribution.

```
1 set.seed(123)
2 x <- rcauchy(1000)
```

The next step is to define a function that implements the Kolmogorov–Smirnov test. This function first sorts the data, then constructs the Empirical Cumulative Distribution Function (ECDF), and finally compares it to the theoretical normal distribution function. The test statistic `D` is defined—as shown by the first equation of the wording—as the maximum absolute difference between the two cumulative distribution functions.

```
1 ks_manual <- function(data) {
2   data <- sort(data)
3   ECDF <- ecdf(data)
4   D <- max(abs(ECDF(data) - pnorm(data)))
5   return(D)
6 }
```

Applying the function `ks_manual` to our simulated data returns:

```
1 > ks_manual(x)
2 [1] 0.1347281
```

Finally, we can assign this value to the object `D`:

```
1 D <- ks_manual(x)
2 D

1 > D
2 [1] 0.1347281
```

Just to make sure we have done things well, we can use the `ks.test()` function as a caution check to see whether our own function that we have elaborated manually is right. This function basically executes the same commands as the one we have elaborated. Importantly, we have to tell R to apply the function to `x` comparing it to a normal distribution.

Lastly, we may validate our manual implementation by comparing it with the `ks.test()` function in R, which performs the same test. Importantly, we have to tell R to apply the function to `x` comparing it to a normal distribution:

```
1 > ks.test(x, "pnorm")
2
3 Asymptotic one-sample Kolmogorov-Smirnov test
4
5 data: x
6 D = 0.13573, p-value < 2.2e-16
7 alternative hypothesis: two-sided
```

As we see that the value of `D` is again 0.13573, we can be sure that our script was correct and well executed.

Since the value of `D` is virtually identical to the one we had obtained previously, we can conclude that our manual implementation of the Kolmogorov–Smirnov test is correct. However, the difference may also indicate some limitations of implementing it manually instead of using the built-in R function.

Question 2

Estimate an OLS regression in R that uses the Newton-Raphson algorithm (specifically BFGS, which is a quasi-Newton method), and show that you get the equivalent results to using `lm`. Use the code below to create your data.

Response

We start by using the code provided to generate the data for this problem. Again, the function `set.seed()` ensures that the same pseudo-random numbers are generated each time the script is executed, which guarantees the replicability of the results.

```
1 set.seed (123)
2 data2 <- data.frame(x = runif(200, 1, 10))
3 data2$y <- 0 + 2.75*data2$x + rnorm(200, 0, 1.5)
```

We now estimate an OLS regression using the BFGS algorithm. The first step is to define a function corresponding to the sum of squared residuals of a linear model (`sosr`). Given a vector of parameters `beta`, the function computes the distance between the observed values and those predicted by the model.

```
1 sosr <- function(beta, x, y) {
2   y_hat <- beta[1] + beta[2] * x
3   sum((y - y_hat)^2)
4 }
```

We then provide initial values for the parameters `beta0` so that the algorithm has a starting point.

```
1 beta0 <- c(0, 0)
```

Finally, we use the BFGS algorithm to find the values of `beta` that minimise the function for the sum of residual squares for linear model:

```
1 algo_bfgs <- optim(beta0, sosr,
2                     x = data2$x,
3                     y = data2$y,
4                     method = "BFGS")
```

The element `"$par"` will contain the values that minimise the function, this is, those that correspond to the estimated OLS coefficients. So calling `"algo_bfgs$par"` returns the esti-

mated coefficients:

```
1 algo_bfgs$par
```

These are the results we obtain:

```
1 > algo_bfgs$par
2 [1] 15.42552234 -0.01739394
```

Finally, we can run an OLS regression model as is typically done in R to check whether we did the process properly.

Lastly, we can compare these results with the standard OLS estimation using the built-in function `lm()` in R:

```
1 > lm_model <- lm(y ~ x, data=data2)
2 > summary(lm_model)
3
```

Table 1:

<i>Dependent variable:</i>	
	<i>y</i>
x	−0.017 (0.188)
Constant	15.426*** (1.137)
Observations	200
R ²	0.00004
Adjusted R ²	−0.005
Residual Std. Error	6.973 (df = 198)
F Statistic	0.009 (df = 1; 198)
<i>Note:</i>	*p<0.1; **p<0.05; ***p<0.01

The results of the OLS regression model are the same we had previously obtained. This means that we have solved the problem correctly.